Root-Cause Analysis of Power Side-Channel Leaks in RISC-V Cryptographic Implementations

Asmita Adhikary, Abraham Basurto-Becerra, Lejla Batina, Ileana Buhan, and Durba Chatterjee

Radboud University, Nijmegen, The Netherlands
firstname.lastname@ru.nl

Abstract. Masking is the standard defense against power-based side-channel analysis (SCA) for cryptographic software, in which sensitive variables are split into independent shares. Although prior work often attributes leakage to microarchitectural effects, architectural interactions alone can already introduce subtle leaks that remain poorly understood. In this work, we propose `ISALeak`, a target-agnostic framework for analyzing full masked implementations to precisely identify and attribute the root causes of side-channel leakage at the instruction-set (ISA) level. `ISALeak` complements statistical tests such as TVLA by not only detecting leakage, but also localizing and explaining its source. We evaluate our approach on masked AES and masked Ascon across multiple compiler versions and optimizations. Using power measurements from ASIC (PicoRV32) and FPGA (Ibex) RISC-V cores, we show that 20-40% of the leaks detected by TVLA for masked AES originate from architectural register interactions. For masked Ascon, 17-23% of the observed leakage likewise stems from ISA-level effects and consistently manifests in physical power traces.

Keywords: Side-Channel Analysis (SCA) · RISC-V · Masking

1 Introduction

Side-channel attacks threaten cryptographic implementations despite their formal security guarantees. Instead of breaking the algorithm, they exploit physical leakages, such as power consumption [21], timing [20], or electromagnetic emissions [13] to recover secrets during execution. In power-based attacks, the net power consumption correlates with processed data, enabling attackers to infer secret-dependent intermediates. Masking is the standard countermeasure against power-based SCA [8]. It splits sensitive values into independent shares to remove leakage from secrets and offers provable guarantees in theoretical models [18]. However, masked software implementations frequently leak in practice. A common cause is unintended interactions between operations: when shares of different variables reuse the same architectural register or memory location, value transitions can cancel the mask and reveal secret-dependent information. Listing 1 shows such a transition-based leak, where loading two masked values

Listing 1 Sensitive variables x and y protected with the same mask m are loaded sequentially into the same register `t3` from memory locations referenced by `t1`. The transition at the architecture level leaks $x \oplus y$.

```
1 lbu t3, 0x0(t1)      # Load x⊕m from address in t1
2 xor t2, t2, a1       # Intermediate instruction
3 xor t6, t2, a2       # Intermediate instruction
4 lbu t3, 0x4(t1)      # Overwrite t3 with y⊕m from t1
```

into the same register exposes $x \oplus y$. These effects typically occur at operation boundaries and are missed by analysis that consider masked gadgets in isolation.

Existing approaches struggle to explain such leaks, often attributing these to the microarchitecture. Formal verification scales poorly to full implementations and abstracts away architecture-dependent behavior [12,18]. Empirical methods such as TVLA [17,24] detect leakage in complete executions, but provide only binary results with little diagnostic insight. Moreover, pipelining, multi-cycle instructions, and oversampled traces obscure the link between measurements and instructions. Consequently, many leaks caused by transitions, glitches, and other processor effects remain difficult to attribute. These limitations leave a key challenge unresolved: *Can we pinpoint the root cause of side-channel leakage in full (masked) software implementations?*

We answer this question affirmatively. Our approach systematically identifies, attributes, and validates leakage sources directly at the Instruction Set Architecture (ISA) level, enabling targeted mitigation early in the development flow [22]. Moreover, it complements statistical techniques such as TVLA by precisely localizing and explaining the root causes of the detected leaks. By analyzing implementations top-down, we expose instruction-level interactions that account for many leaks observed in practice, before lower-level microarchitectural effects are considered. Early attribution turns opaque leakage measurements into actionable fixes, enabling developers to eliminate software vulnerabilities.

Related work. Papagiannopoulos et al. [29] first highlighted the impact of ARM processor effects on side-channel leakage, and later studies uncovered additional leakage sources across devices [3, 26, 28]. Building on these observations, several subsequent work systematically characterized microarchitectural effects on ARM processors [14, 26, 29, 36, 37], and developed simulators and profiling tools that explicitly model such behavior. For example, ELMO [27] models power consumption as a linear combination of intermediate values and transitions, later extended to capture multi-cycle interactions [31]. In contrast, ABBY [5] profiles target devices and leverages machine learning to approximate leakage, extending support to Cortex-M0 and M3. A broader overview of such approaches is provided in [7]. However, because these tools explicitly model device-specific microarchitectural behavior, they are limited to the supported platforms.

Most tools for hardening masked software implementations target ARM architectures [36, 37]. Rosita [30, 32] attempts automated mitigation by exploring

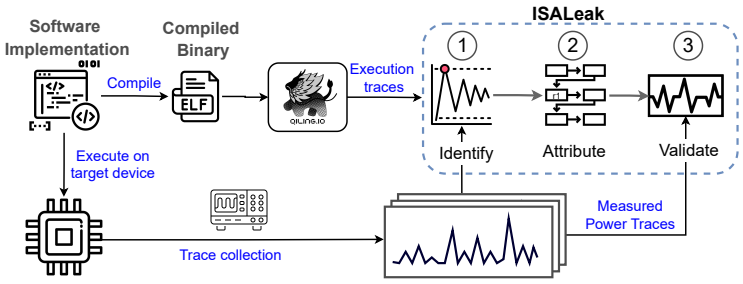


Fig. 1: Leakage Analysis of masked implementations using `ISALeak`

random instruction substitutions to eliminate leaks. However, without precise attribution of the underlying cause, these approaches operate heuristically and may introduce unnecessary overhead or fail to address the true source of leakage.

Support for RISC-based platforms has only recently begun to emerge. COCO [15] and aLEAKator [2] focus on probing-based verification at the gadget and full-implementation levels, respectively, but provide limited insight into the root cause of detected vulnerabilities. While ARCHER [1] detects architectural leakages on RISC-V cores and OoOLyzer [9] extends architectural leakage analysis to out-of-order RISC-V cores by leveraging fine-grained execution traces to identify microarchitecture-induced violations of masking assumptions, both stop short of precise, instruction-level attribution. To address these limitations, we introduce `ISALeak`, built on ARCHER [1], a new class of tools that leverages dynamic instrumentation to precisely localize and explain leakage at the ISA level, enabling targeted and effective mitigation for RISC-based architectures.

Target audience. `ISALeak` is aimed at developers and security evaluators who optimize cryptographic implementations and evaluate the impact of SCA leaks.

2 Background

Masked AES. We analyze a first-order Boolean-masked implementation of AES [10] from the open-source repository by CENSUS [1], based on the masking scheme described in [25]. Masking is applied to plaintext, key schedule, and round functions, using six independent masks: m_1, m_2, m_3, m_4, m, and m', which are propagated and updated across AES operations as depicted in Figure 2.

AddRoundKey. The key bytes are XOR-ed with the masked state, where each byte of the state is masked with a common mask m.

SubBytes. Being the only non-linear operation, s^m is implemented as a masked table lookup: $s^m(x \oplus m) = s(x) \oplus m'$, thus updating mask m with m'.

ShiftRows. This operation rearranges the positions of the bytes in the state, thereby preserving mask m'.

[1] `https://github.com/CENSUS/masked-aes-c`

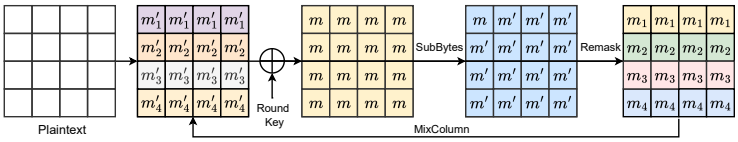


Fig. 2: Application of masks in the execution of first-order masked AES

MixColumns. The four rows of the state matrix are assigned: m_1, m_2, m_3, m_4, having derived from m'. The outputs are masked with m'_1, m'_2, m'_3, m'_4, which are reused as the input masks for the next round, minimizing recomputation.
Final Round. The final AddRoundKey step includes a mask removal process that cancels the active masks and yields the correct unmasked ciphertext.

Masked Ascon. We analyze a first-order Boolean-masked implementation of Ascon-128 authenticated encryption with associated data (AEAD) [11], adapted from an open-source repository[2]. We chose this implementation because Ascon's permutation-based, sponge-style [6] design offers insights into the masking behavior of a structurally different cipher, as compared to AES. Furthermore, this implementation uses inline assembly to implement the permutation operation.

Ascon is bit-sliced by design and processes a 320-bit state, split into 5 words of 64-bits each, labelled as x_0, x_1, x_2, x_3 and x_4, comprising a 128-bit key, 128-bit nonce, 64-bit associated data, and 64-bit plaintext to produce an authenticated ciphertext of the same length as the plaintext, along with a 128-bit tag. Ascon uses a sponge-based construction and applies the permutation across four phases: initialization, associated data absorption, plaintext encryption, and finalization. It applies a 12-round SPN-based permutation during initialization and finalization and a 6-round permutation during associated data and plaintext processing.

Due to the absence of a reference masked Ascon implementation in either a high-level language or RISC-V assembly, we derive a masked RISC-V implementation by transforming an existing protected ARM implementation[3]. The transformation adapts the inline ARM assembly to inline RISC-V assembly, guided by an existing unprotected RISC-V implementation[4]. The masking scheme from `protected_bi32_armv6` [16] is retained, while dynamic memory allocation is replaced by static allocation. Compared to the original ARM version, the RISC-V implementation differs in the use of inline RISC-V assembly, static memory allocation, and standard libraries to ensure constant-time behavior.

The 2-share masked implementation processes a 640-bit state and applies a 128-bit mask to the key, a 128-bit mask to the nonce, a 128-bit mask to a 128-bit

[2] `https://github.com/ascon/simpleserial-ascon`
[3] `https://github.com/ascon/simpleserial-ascon/tree/main/Implementations/crypto_aead/ascon128v12/protected_bi32_armv6`
[4] `https://github.com/ascon/ascon-c/tree/main/crypto_aead/asconaead128/asm_rv32i`

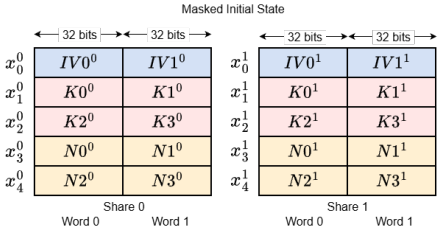


Fig. 3: State representation of 2-share masked Ascon where *Share 0* and *Share 1* hold the computed shares and the mask itself, respectively.

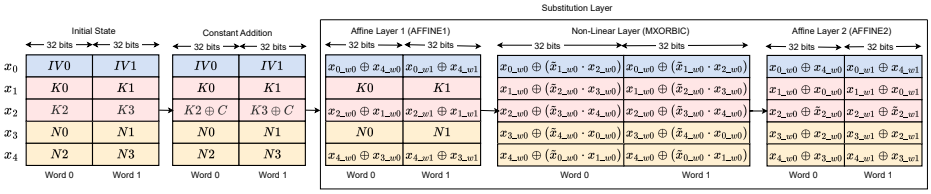


Fig. 4: Permutation of Ascon - constant addition and S-Box operation

plaintext (we consider 128-bit plaintext for our experiments), and a 64-bit mask to the associated data, as illustrated in Figure 3.

All inputs are Boolean-masked and processed as shares throughout the round function. In the 2-share implementation, each 32-bit state word is split into a data share and a random mask share. While each round comprises three operations, our masking analysis focuses on the S-box and linear diffusion layers (Figure 4).
Addition of Round Constant (p_C). In each round i, the constant c_i is XOR-ed into the 64-bit register x_2. In the unmasked setting, the state is represented as $S = x_0\|x_1\|x_2\|x_3\|x_4 = IV\|K\|N = IV\|K0\|K1\|N0\|N1$. For the 2-share masking scheme, $S = (x_0^0\|x_1^0\|x_2^0\|x_3^0\|x_4^0) \oplus (x_0^1\|x_1^1\|x_2^1\|x_3^1\|x_4^1)$.
Masked Substitution Layer (p_S). A 5-bit S-box $S(x)$ is applied in parallel to each bit-slice of the five state registers $x_0, \ldots, x_4$. The implementation of the substitution layer is made up of three operations, the first linear `AFFINE1` layer, the non-linear `MXORBIC` layer (or, the S-Box) and the second linear `AFFINE2` layer.
Masked Linear Diffusion Layer (p_L). Being linear, each share of x_i is independently updated by applying the same set of rotations and XORs, using $\Sigma_i(x_i)$.

3 ISALeak

In this section, we present an end-to-end target-agnostic framework to **identify**, **attribute**, and **validate** sources of power-based side-channel leakage in full software implementations of masked algorithms on RISC-V. Operating at the instruction set architecture (ISA) level, `ISALeak` bridges the gap between empirical leakage detection and instruction-level root-cause attribution. The framework comprises three main components that together form a complete leakage analysis flow from detection to validation, as illustrated in Figure 1. Since the

analysis is performed directly on the compiled binary, ISALeak can be applied both in the pre-silicon stage and alongside physical evaluation.

Leakage Modeling. To systematically analyse data-dependent leaks at the ISA level, ISALeak models a program execution and its potential leakage sources in a unified, architecture-agnostic manner. We represent a software program or its compiled binary as a finite sequence of assembly instructions $\mathcal{P} = (I_0, \ldots, I_{|\mathcal{P}|-1})$, where each instruction I_t operates on architectural registers $\mathtt{r_0} - \mathtt{r_{31}}$ (each of width W[5]), memory locations, or constants. The state of a register $\mathtt{r_i}$ after executing instruction I_t is denoted as r_i^t. The execution of $\mathcal{P}$ on an input x yields a register-state trace $\mathsf{Trace}(x) = \langle S_0, S_1, \ldots, S_{|\mathcal{P}|-1} \rangle$ where each $S_t = \{r_0^t, r_1^t, \ldots, r_{31}^t\}$ represents the complete state of the architectural register after instruction I_t, for a 32-bit architecture processor.

Leakage Functions. ISALeak extends the CPU-independent leakage model [37] to the architectural level by associating a leakage function with each instruction and register update. The framework works with two fundamental classes of leaks used in the literature [4].

1. *Value-based* leakage model captures leaks from the value stored in a register after executing instruction I_t, denoted $l_{\text{val}}(r_i, t) = f(r_i^t)$, where $f(\cdot)$ is a value-based leakage model such as the Hamming weight or the identity function [4]. The cumulative value-based leakage for instruction I_t is given by $L_{\text{val}}(t) = \sum_{i=0}^{31} l_{\text{val}}(r_i, t)$. This formula provides an aggregate value for the data-dependent component of the net power consumption (refer to the power consumption model in [25]), by tracking register values.
2. *Transition-based* leaks arise when the register $\mathtt{r_i}$ is overwritten by instruction I_t. The leak depends on the data transition from its previous to its new value: $l_{\text{trans}}(r_i, t) = g(r_i^{t-1}, r_i^t)$, where $g(a, b)$ may represent a transition-based model such as the Hamming distance $\text{HW}(a \oplus b)$ [4]. The total transition-based leakage for instruction I_t is $L_{\text{trans}}(t) = \sum_{i=0}^{31} l_{\text{trans}}(r_i, t)$. This component models power drawn during register updates, capturing leakage from transitions, glitches, or bitline switching activity.

Integration within ISALeak. These leakage functions form the foundation of the ISALeak framework, which are used by the different components, including *identification*, and *leakage attribution*. Per-instruction-level leakage models enable ISALeak to perform detailed attribution of observed power variations to their underlying instruction- and register-level changes.

3.1 Identify Leaks (Detection)

The goal of the first stage of ISALeak is to identify instructions in a full implementation that show side-channel leaks. This stage operates on instruction-accurate simulated traces generated from the same compiled binary executed on the target device [1].

[5] W is 32 bits for a 32-bit architecture

Generation of execution traces. The first step of the component is to generate architecture-level execution traces, for which we employ Qiling, a unicorn-based emulation framework, inspired by the setup used in a recent work on RISC-V implementation evaluation [1]. The setup takes as input the `.elf` file of the target implementation, along with the corresponding inputs (such as plaintext, key, nonce, associated data, and masks). The execution trace comprises the sequence of executed assembly instructions and the architectural register contents (including the program counter and stack pointer) after each instruction.
Generation of simulated power traces. Next, a leakage model function is applied to the execution traces to generate *simulated power traces*. We use both value-based and transition-based leakage models. For value-based leakage, we use the Hamming Weight (HW) model; for transition-based leakage, we use the Hamming Distance (HD) model. The HW/HD functions can also be applied to specific bit(s) of the register. Each simulated power value corresponds to a single assembly instruction, thereby aligning the traces to code execution. Depending on the chosen leakage model, we obtain either an HW-trace or an HD-trace.
Leakage Assessment is performed on the simulated traces using standard non-specific tests such as the fixed-vs-random TVLA [17] and the shares-TVLA [24], depending on the masking order under evaluation. For simulated traces, each sample maps directly to a specific assembly instruction, so significant $|t|$-values indicate potentially leaking instructions. However, TVLA on simulated traces may produce *false negatives*: because these traces abstract overall power consumption, leakage can be masked when the fixed input lies close to the mean of the random distribution, yielding near-zero mean differences.

To mitigate these limitations, we complement TVLA with correlation-based leakage detection. For *value-based leaks*, we correlate simulated HW-traces with intermediate values; a non-negligible Pearson correlation indicates that the value is present in a register during the corresponding instruction. For *transition-based leaks*, which are common in masked implementations, we instead correlate HD-traces with masking-derived intermediate combinations, since HD-traces capture interactions between consecutive instructions. The calculation of HD contributions per register further enables the detection of leaks across non-consecutive instructions by isolating each register's effect before aggregation.

The outcome of this stage is a set of instruction indices and potential data interactions that exhibit a strong statistical correlation with simulated traces. These candidate instructions or data-interaction hypotheses are then forwarded to the next stage for detailed attribution and root-cause analysis.

3.2 Attribute (Data Interaction Analysis)

The second stage of `ISALeak` assigns the detected leakage points to operations and data interactions within the implementation, pinpointing the exact source of leaks. Tracing the source of value-based leaks involves tracing the register contents in the execution trace backwards recursively, starting from the detected instruction to the previous instructions that modified/set the content of the source

operand(s). On the other hand, understanding the cause of *transition leaks* requires recursive tracing of both source operands and the register contents before the instruction, since the leak depends on consecutive register states. For each *leaky* instruction I_i : (`OP` $\mathtt{r_d^i}$, $\mathtt{r_m^i}, \mathtt{r_n^i}$) that modifies the destination register r_d^i, using the values contained in r_m^i and r_n^i (operands one and two) using operation `OP`, the framework performs a backtracking on the *execution trace* to determine the *sensitive data* that leaks.

We trace *the destination* and *source registers* through preceding instructions until we resolve the content of a register (i.e., map the register value back to a sensitive variable). *Attribution* requires the following actions:

1. Find instruction I_x that modified the content of register r_n^i (operand one of the leaky instruction I_i), such that $r_d^x = r_n^i$, where $x < i$;
2. Resolve the content of the registers r_n^i, in terms of (combination) of sensitive variables of the executed algorithm.
3. Find instruction I_y that modified the content of register r_m^i (operand two of the leaky instruction I_i), such that $r_d^m = r_n^i$, where $y < i$;
4. Resolve the content of the registers r_m^i, in terms of (combination) of sensitive variables of the executed algorithm.

The steps described above are repeated until we can unequivocally resolve the contents of the registers of interest. Two challenges arise when *resolving the contents of a register*:

1. *Transformations:* the sensitive variable may be altered through representation changes (encodings, bitslicing), obfuscation (masking), or modification due to internal algorithmic updates.
2. *Memory addressing:* implementations may store values at computed offsets (e.g., S-box lookups), requiring analysis of how data are organized and accessed in memory to identify which sensitive variable is loaded.

The result is a data-dependency path linking the leaking instruction I_i to a (transformed) sensitive variable. To improve efficiency, the execution trace is transformed into a *symbolic trace*, where register values are replaced with labels denoting sensitive variables or intermediates. Each label denotes the value held by a register, such as associated data, a key, or mask bytes/words. Each label captures the semantic meaning of a register value, such as associated data, key material, or mask words. Label generation requires identifying the corresponding register locations and indices, and mapping observed hexadecimal values to algorithm-specific intermediates. This abstraction enables efficient pattern matching and identification of interactions between shares that violate masking assumptions.

The final step of attribution is to determine which masking scheme condition is violated and caused the leaky instruction. This step requires knowledge of the masking scheme, including its order and the corresponding masked operations. Using this information, `ISALeak` detects violations in which registers hold shares of the same variable, or in which shares masked with the same randomness are reused or combined, leading to mask cancellation and unintended data disclosure.

Table 1: Number of assembly instructions for Masked AES and 2-share Masked Ascon

	Masked AES					Masked Ascon (2-shares)[6]				
	`-O0`	`-O1`	`-O2`	`-O3`	`-Os`	`-O0`	`-O1`	`-O2`	`-O3`	`-Os`
v10.2	48066	12300	11364	9858	15061	-	26049	33253	33246	27527
v12.2	48545	12295	11660	9532	15056	-	25588	32811	32451	27706
v13.2	48728	11996	11342	8686	14783	-	25592	32809	32388	27694
v14.2	49212	12436	11385	7466	15097	-	-	-	-	-

The output of this stage is a list of attributed leakage points, each of which links an instruction or a pair of instructions to a specific variable interaction. These results are also provided to the developer, revealing how compiler decisions, such as instruction sequences and register usage, can lead to unintended information flow in high-level implementations. This is also a valuable tool for evaluating manually written assembly code. Subsequently, these results are validated using correlation analysis.

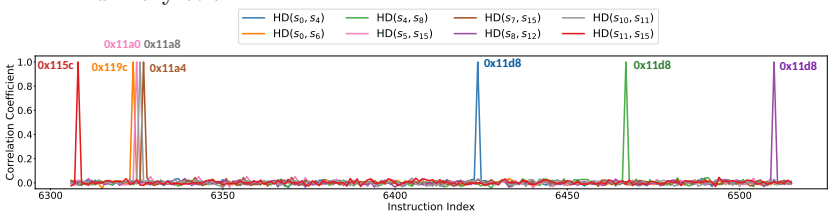
3.3 Validation (Correlation Analysis and Confirmation)

In the final stage, `ISALeak` validates the leakage hypotheses derived from the attribution phase by performing correlation analysis on *measured (power) traces*. For each attributed instruction (pair), the corresponding intermediates or register transitions are correlated with measured traces. Significant correlation values confirm that ISA-level data interactions manifest as observable side-channel leakage in physical measurements. Comparing these results with TVLA enables precise attribution of the leaks source. The output is a set of validated leakage sources, enabling precise,code-level mitigation. At this step, multiple correlation peaks are may be observed for the same interaction, as a single leakage source can manifest at several sample points in the measured power trace [22].

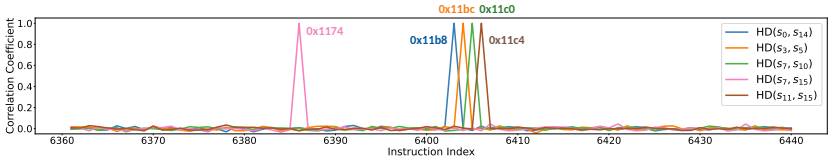
In the following sections, we employ `ISALeak` to systematically analyze how compiler optimizations influence data interactions and, consequently, power-based side-channel leakage in full implementations of masked AES and masked Ascon. Table 1 summarizes the instruction counts for masked AES and masked Ascon across compiler versions and optimization levels, highlighting that compiler version and optimization level introduce changes in the compiled binary.

Hardware Setup. For practical evaluation, we use an ASIC implementation of PicoRV32 [19, 35] and an FPGA implementation of Ibex [23] cores, both supporting a 32-bit RV32IMC instruction set. PicoRV32 has no pipelining; Ibex implements a 2-stage pipeline, thereby evaluating the performance of `ISALeak` across different microarchitectures. For trace acquisition, we use ChipWhisperer-

[6] Compiling Masked Ascon for -O0 (all GCC versions) and for GCC v14.2 (all optimization levels) requires modifying the inline assembly instructions considerably (while keeping it functionally equivalent), thus resulting in non-identical binaries, hence the omission from the table.



(a) GCC Version v13.2, Optimization level −O2



(b) GCC Version v14.2, Optimization level −O2

Fig. 5: Correlation analysis results on HD simulated traces highlighting SubBytes output interactions in Masked AES across two different compiler versions.

Husky[7], which uses synchronous sampling and generates a 5 MHz clock signal for the PicoRV32 core, while the Ibex core is operated at 100MHz.

4 Case Study: Masked AES

We evaluate a first-order masked AES implementation compiled with the RISC-V GNU toolchain at different optimization levels: `-O0`, `-O1`, `-O2`, `-O3`, and `-Os`. Since each level alters register allocation, instruction scheduling, and data transformations, we treat them as distinct implementations. We repeat the analysis across four compiler versions: `GCC v10.2`, `v12.2`, `v13.2`, and `v14.2`. For brevity, we explain the steps using only one implementation per step.

4.1 Identification

Simulation power traces are generated individually for different optimization levels and compiler versions. The first step is to test for *unprotected key-dependent variables* in the registers. For this, we use the HW simulation traces and correlate them with the interim unprotected variables. Next, to test for transition-based leaks, we identify which AES state bytes share the same mask from the masking scheme depicted in Figure 2. For instance, all SubBytes output bytes are protected by the same random mask (m'). In such a scenario, interactions between any two state bytes would violate the statistical independence guaranteed by masking. Likewise, in the `Remask` operation, the AES state is re-masked using four random bytes (m_1, m_2, m_3, m_4), each applied to one row of the state matrix. Herein, data interactions among these bytes can introduce unintended

[7] `https://rtfm.newae.com/Capture/ChipWhisperer-Husky/`

leakage. Interestingly, this is implementation agnostic and hence applicable to all compiled binaries.

Using this information, we construct an exhaustive set of possible byte interactions. The next step involves preparing the data to be used for correlation. This involves computing intermediates for every input used for each trace. We perform correlation across 5 000 simulated traces generated from program executions using random plaintexts and masks under a fixed key. Figure 5 presents the correlation results for GCC versions 13.2 and 14.2, both compiled with optimization level `-O2`. Although the correlation is computed over all possible byte combinations, only interactions that exhibit noticeable peaks are shown. Each peak is annotated with the corresponding program counter (PC) value of the instruction responsible for that event. In particular, despite identical optimization levels, PC values differ between compiler versions, indicating variations in instruction scheduling and register allocation, demonstrating that compiler version is an important factor influencing SCA leaks. The exhaustive list of byte interactions that exhibit statistically significant correlation peaks is listed in the Appendix (refer to Table 6).

4.2 Attribution

In this section, we first examine how compiler optimizations influence the register allocation and instruction scheduling, and use them to explain the leaks identified in the previous section.

Instruction Scheduling: `remask` Function in Masked AES. The `remask` function is responsible for masking various elements: it is invoked to mask the round keys, the plaintext, and the input to the `MixColumns` operation. In this section, we focus specifically on its invocation prior to the `MixColumns` operation (see Figure 2). As shown in Listing 2, the function takes as input the state and eight mask bytes, a combination of the initial masks, and derived values. The first four bytes represent the new masks applied to each row of the state, while the last four bytes are used to remove the previous masks. For space reasons, we detail the `remask` function under two optimization levels: `−O1`, where the function is not inlined, and `−O2`, where it is inlined.

Masked AES -O1(GCC v13.2). Table 2 provides a visual representation of the sequence of operations. The state address is passed through the register `a0`, and the new mask bytes m_1–m_4 are loaded into registers `a1`–`a4`, while `a5`–`a7` contain the old mask value m', applied to the output of the S-box. Before updating the mask, the compiler computes the XOR between the old and new masks, $m_i \oplus m'$ for $i = 1, \ldots, 4$, storing the results in successive PC addresses starting from `0xb00`. The core operations employ register `t3`, beginning with loading `s[0]` (`0xb10`), applying the XOR with mask from `a1` (`0xb14`), and writing the result back (`0xb18`). This procedure is systematically repeated for all four columns. The `remask` function exhibits a consistent implementation across the three compiler versions evaluated at the `−O1` optimization level.

Listing 2 During each round, the `remask` function updates the state by removing the S-box mask m' and applying new masks m_1, m_2, m_3, m_4 for every row of the state. When called before `MixColumns` operation, m_5, m_6, m_7, m_8 equals m'.

```
1   for (int i = 0; i < 4; i++) {
2     (*s)[i][0] = (*s)[i][0] ⊕ (m1 ⊕ m5);
3     (*s)[i][1] = (*s)[i][1] ⊕ (m2 ⊕ m6);
4     (*s)[i][2] = (*s)[i][2] ⊕ (m3 ⊕ m7);
5     (*s)[i][3] = (*s)[i][3] ⊕ (m4 ⊕ m8);
6   }
7 }
```

Table 2: Register state during the execution of `remask` at −`O1` (GCC v13.2). This involves updating state ($^*s$) by replacing old mask shares with new ones. The rows correspond to the registers used for passing arguments to `remask` as outlined in Listing 3. Each column corresponds to one instruction specified by the program counter values. The expression in each cell indicates the first time the value is loaded into the register. Grey cells indicate a store operation of the register content.

	Function Arguments	0xafc	0xb00	0xb04	0xb0c	0xb10	0xb14	0xb18	0xb1c	0xb20	0xb24	0xb28
a0	$^*s$											
a1	m_1	$m_1 \oplus m'$										
a2	m_2		$m_2 \oplus m'$									
a3	m_3			$m_3 \oplus m'$								
a4	m_4				$m_4 \oplus m'$							
a6	m'											
t3						$s[0] \oplus m'$	$s[0] \oplus m_1$		$s[1] \oplus m'$	$s[1] \oplus m_2$		$s[2] \oplus m'$

Masked AES -O2 (GCC v14.2). For clarity, Table 3 provides a visual representation of the sequence of operations. The general structure of the computation is similar to that observed under −`O1`: the compiler first pre-computes the combination of the old mask (m') with the new masks m_1, m_2, m_3 and m_4. However, unlike the −`O1` case, this pre-computation is performed only once, and the resulting values are retained in dedicated registers throughout the execution of the ten rounds. Before the `MixColumns` operation, the state bytes of the first column are loaded from memory (addresses `0x11b8-0x11c4`), in registers `a0,a2,a3,a4`, respectively. At this point, the state bytes are masked with m' due to the preceding `ShiftRows` operation, and the registers hold (some) of the output of the `ShiftRows` operation. The loaded bytes are then XORed with the precomputed masks (addresses `0x11c8-0x11d4`), and the results are written back to memory. The ordering of operations differs from that in the −`O1` implementation: the state bytes are first loaded, then the new masks are applied, and finally, the masked values are stored. The column pre-loaded in Table 3 shows the register contents preceding the state bytes loading.

When considering the different compiler versions, we notice subtle differences in how the `remask` function is implemented. The differences are related to which registers are used for the additional operations. For instance, in version 13.2 and

Table 3: The execution of `remask`, (GCC v14.2, `-O2`) updates the state by removing the old mask and applying a new one. The program counter values index the operations. The corresponding instruction sequence is described in Listing 4.

	Pre-loaded	0x11b8	0x11bc	0x11c0	0x11c4	0x11c8	0x11cc	0x11d0	0x11d4
t0	$m_1 \oplus m'$								
t2	$m_2 \oplus m'$								
a1	$m_4 \oplus m'$								
t6	$m_1 \oplus m'$								
a0	$s_{14} \oplus m'$	$s[0] \oplus m'$				$s[0] \oplus m_1$			
a2	$s_3 \oplus m'$		$s[1] \oplus m'$				$s[1] \oplus m_2$		
a3	$s_7 \oplus m'$			$s[2] \oplus m'$				$s[2] \oplus m_3$	
a4	$s_{11} \oplus m'$				$s[3] \oplus m'$				$s[3] \oplus m_4$

12.2 the compiler uses registers `a1-a4` for performing the operations, instead of `a0,a2-a4`, and keeps the XOR values between the old and new masks in different registers (`t0,t2,s1,t3` for 13.2 and `a7,t1,t6,s2` for 12.2)

Explaining leaks. Section 4.2 describes the data interactions and register usage in the `remask` operation. We identify five leaky instructions within this operation, one at index 6385 and four consecutive instructions at indices 6402–6405. The corresponding program counters (PCs) are highlighted in Figure 5, which shows the correlation HD-traces . The instructions at indices 6402–6405 lie within the `remask` function and align with the PCs listed in Table 3. Each of these instructions corresponds to a byte-level transition between specific output positions of the SubBytes operation:(14,0), (3,5), (7,10), and (11,15).

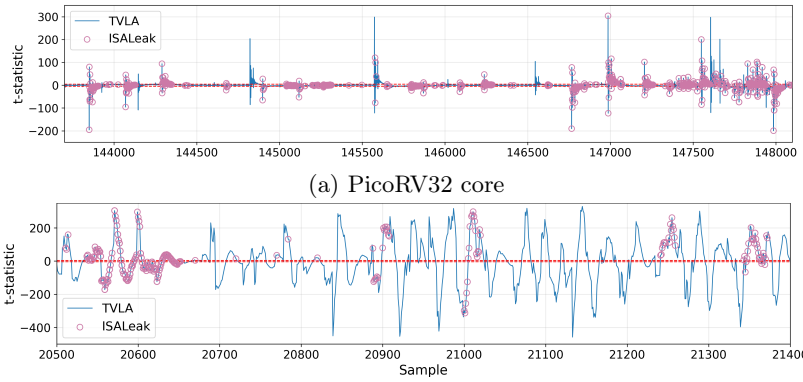
To explain the root cause of these estimated leaks, we track the register content before and after the execution of each instruction from Table 3.

Leak 1: Instruction at PC `0x11b8` (Index 6385). This instruction is the first operation in the `remask` function and updates register `a0` with the transition from $s_{14} \oplus m' \rightarrow s_0 \oplus m'$. As a result, the register transition reflects the value $s_0 \oplus m' \oplus s_{14} \oplus m' = s_0 \oplus s_{14}$, removing the common mask m' and leaving a transition purely dependent on the difference between two sensitive state bytes. This causes a statistically significant correlation between data $s_0 \oplus s_{14}$ and the simulated traces obtained using the HD leakage model, explaining the corresponding peak in Figure 5.

Leaks at PC `0x11bc`, `0x11c0` and `0x11c4`. These leaks can be explained by examining the transitions in registers `a2`, `a3`, and `a4`, as shown in Table 3. Specifically, these registers undergo updates that result in net transitions corresponding to $s_3 \oplus s_5$, $s_7 \oplus s_{10}$, and $s_{11} \oplus s_{15}$, respectively. Each of these transitions reflects a direct interaction between sensitive SubBytes outputs due to register reuse, leading to observable leakage in the simulated correlation traces.

4.3 Validation

In this step, we correlate measured traces with the identified byte interactions. Figure 6 depicts annotated TVLA traces obtained from PicoRV32 and Ibex



(a) PicoRV32 core

(b) Ibex core

Fig. 6: Masked AES (1st Round): TVLA and correlation (GCC v10.2, `-Os`)

cores. The peaks highlighted with pink circles indicate the samples where the leaks are successfully resolved at the architecture-level using `ISALeak`. Note that we plot only the first round for a clear indication of the peaks. The evaluation extends to full implementations. Table 4 summarizes the total number of peaks that were detected by TVLA and the number of peaks explained using `ISALeak`. The first round of masked AES implementation takes 4000 sample points on PicoRV32, whereas on Ibex, it needs only 900 sample points. The number of explained leaky sample points depends on the possible combinations of the data-interactions identified in the first step of `ISALeak`. For byte-masked AES implementation, we can exhaustively list all possible byte-level interactions that can potentially violate masking assumptions. This varies with the target cryptographic algorithm. From the results on PicoRV32, in Table 4, we see that `ISALeak` can resolve 45% (460 out of 1022) of TVLA leaks, whereas for Ibex, 22% leaks are resolved. Moreover, `ISALeak` reports potential leaks from unwanted byte interactions in architectural registers, which TVLA fails to capture. For these sample points, the $|t|$-value is less than 4.5, and thus the circles are clustered close to the x-axis, such as the sample points around 145 000, 145 750, 146 250, and 147 500 in Figure 6a.

We validate the presence of *transition-based* leaks in `remask` operation presented in Section 4.2, using real traces collected from PicoRV32 for versions v13.2, and v14.2, for optimization levels `-O2`. We correlate the real power traces with the HD of two S-box output bytes, from the same row of the state matrix. We repeat the correlation for all row-wise byte pair combinations, and we observe clear peaks for several of the byte pairs. The correlation is performed over 10 000 traces, where each power trace is obtained for a distinct plaintext and mask. For v13.2, we statistically significant correlation for 8 distinct byte interactions, whereas for version 14.2, we observe only 5 interactions, corroborating with the correlation results on simulated traces (refer Figure 5). The correlation plots for other levels are presented in Figure 9.

Table 4: TVLA and `ISALeak` evaluation results for Masked AES (1st Round)

PicoRV32 (over 4000 samples)

		`ISALeak`	
		Leak	No Leak
TVLA	Leak	460	562
	No Leak	404	2775

Ibex (over 900 samples)

		`ISALeak`	
		Leak	No Leak
TVLA	Leak	190	664
	No Leak	11	36

5 Case Study: Masked Ascon

In this section, we systematically analyze a two-share masked Ascon implementation, which inherently follows a bit-sliced design, using `ISALeak`. We consider three compiler versions, GCC `v10.2`, `v12.2`, and `v13.2`, from the RISC-V toolchain.. Each implementation is compiled with the four optimization levels: `-O1`, `-O2`, `-O3`, and `-Os`.

5.1 Identification

Based on the masking scheme described in Section 2, *no interaction should occur between the two shares of any sensitive variable*, as that would violate the non-completeness property fundamental for this implementation. Conversely, operations that manipulate multiple subwords or temporary variables derived from the same masked word—such as rotations, word mixing, or bitwise compositions—reuse the same mask by design. This reuse does not inherently compromise security, as long as the compiler preserves the independence between masked shares. This observation remains implementation-agnostic and is thus applicable to all compiled masked Ascon binaries across optimization levels.

Having established that the *recombination of two shares of the same variable can reveal parts of the original unmasked value*, we construct a set of *native* and *sensitive* variables from the permutation operation of an unprotected Ascon implementation. These unmasked intermediates act as reference points to identify potential leakage sources in the masked implementation. From the state representation of unprotected Ascon illustrated in Figure 4, we observe that the non-linear substitution layer comprises three operations—(1) `AFFINE1`, (2) `MXORBIC`, and (3) `AFFINE2`. We simulate the outputs of each of these operations from the unprotected implementation, as they represent either *native variables* or linear/non-linear combinations of *sensitive variables*. We also take into account that masked Ascon represents all state variables in a bit-sliced form and separates each 64-bit input word into two 32-bit halves containing the even (planeE) and odd (planeO) bit positions, respectively. For instance, after `AFFINE1`, $affine1\_x_3$ holds $\{N0, N1\}$, while after `MXORBIC`, $mxorbic\_x_1$, holds a non-linear combination of $\{N0, N1\}$ and $\{K0, K1\,K2, K3\}$. We simulate a selection of *sensitive variables*:

1. **Input:** $nonce\_word\{0,1\}\_plane\{E,O\}$

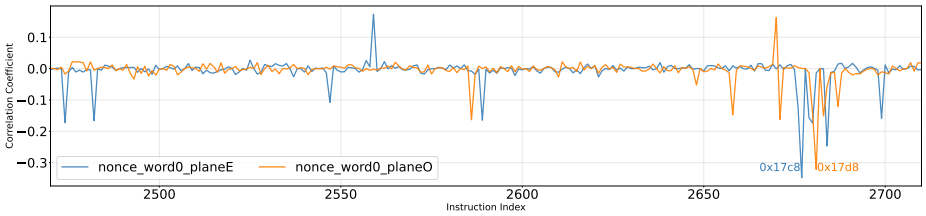


Fig. 7: Correlation analysis for HD-traces, masked Ascon, GCC v10.2, `-O1`.

2. **From `AFFINE1`:**
   $affine1\_\{x_0,\ldots,x_4\}\_plane\{E,O\}$, $affine1\_\{x_0,\ldots,x_4\}\_word\{0,1\}$
3. **From `MXORBIC`:**
   $mxorbic\_\{x_0,\ldots,x_4\}\_plane\{E,O\}$, $mxorbic\_\{x_0,\ldots,x_4\}\_word\{0,1\}$
4. **From `AFFINE2`:**
   $affine2\_\{x_0,\ldots,x_4\}\_plane\{E,O\}$, $affine2\_\{x_0,\ldots,x_4\}\_word\{0,1\}$

To identify leaks, we perform correlation over 10 000 simulated traces of the masked implementation with the *constructed variables*. The analysis is performed for all binaries compiled for different compiler versions and optimization levels. The simulated traces are generated using fixed key and random nonce, associated data, plaintext along with all random masks. No information related to the masks is used in these experiments.

Figure 7 illustrates the correlation peaks of masked Ascon having functions annotated for GCC version 10.2 at optimization level `-O1`, in `generate_shares` and `P2` functions. The `generate_shares` function (from index 317) is responsible for generating the masked shares of the inputs: key, nonce, associated data and plaintext, and deals with unprotected *native variables*, and so, it is expected to show peaks until the variables are masked. Then, `ascon_initaead` starts the initialization phase wherein `TOBI` loads the even and odd bits of the masked shares of the `native variables`. The leaks we observe at `P2` (from index 2402) are unintended leaks since the variables have been masked at this point. Figure 7 shows the zoomed-in correlation peaks for the same, with two highest peaks annotated with PCs. Our framework identifies multiple leaky instructions as shown in Figure 7. We focus on the root cause analysis of the two leaky instructions in `P2`, for GCC `v10.2` optimization level `-O1`, at **PC** `0x17c8` and **PC** `0x17d8` with correlation peak of 0.34 and 0.32 respectively, with respect to $nonce\_word0$.

5.2 Attribution

In this section, we examine how compiler optimizations influence register allocations and instruction scheduling, even for manually written assembly code, resulting in unintended interaction between *sensitive variables*. We focus on the two leaks identified in Section 5.1.

Register Allocation: P2 Function in Masked Ascon The `P2` function is responsible for performing the masked permutation operation throughout masked Ascon. `P2` gets inlined and all its subsequent called functions (`AFFINE1, MXORBIC, AFFINE2, LINEAR`) being forceinlined, behaves in a similar manner for all the optimization levels. The two leaky instructions occur at the very beginning of the `LINEAR` diffusion function inside `P2`. Since, the round permutation in masked Ascon is written in assembly, it shows little to no difference between the different compiler versions and optimization levels, and the leaky points stay consistent for the different compiler versions and optimization levels. We focus on the two leaky instructions when compiled with GCC version 10.2, for optimization level `-O1`.

`P2` and all subsequently called functions (`AFFINE1, MXORBIC, AFFINE2, LINEAR`) are force-inlined and exhibit similar behavior across optimization levels, propagating bit dependencies. Concretely, `a2` feeds a 32-bit logical right shift by 5 (written to `s11` at `0x1788`), while `t6` and `t1` feed 32-bit values that are left shifted by 25 before being written to `s11` at `0x17c4` and `0x17d4`, respectively. These three writes are part of the same unrolled `LINEAR` step and land in the *same* destination register (`s11`) in back-to-back instructions[8].

Explaining Leaks To explain the root cause of the two leaks identified above, we track the register contents before and after each instruction.

Leak 1: Instruction at PC `0x17c8` (Index 2679). The effect of an executed instruction shows up in the next instruction in the execution trace. Hence, we track the destination register of the previous PC, i.e., 0x17c4 (index 2677) where register `s11` gets updated. Immediately before `0x17c4`, `s11` holds the value written at `0x1788`; at `0x17c4` it is overwritten. Register `s11` was previously holding `a2` $\gg$ `5` at PC 0x1788 and gets overwritten with `t6` $\ll$ `25` at PC 0x17c4. On expanding the register contents, `s11` is updated:

PC 0x1788: $[K1^1 \oplus (N1^1 \cdot \tilde{R}OL5(K0^1 \oplus K2^1)) \oplus (N1^1 \cdot (K1^1 \oplus K3^1)) \gg 5]$

PC 0x17c4: $[((N1^0 \oplus N3^0) \cdot \tilde{R}OL5(N0^0)) \oplus ((N1^0 \oplus N3^0) \cdot N1^0) \ll 25]$.

This transition overwrites in `s11` a *Share 1*–derived expression that mixes key and nonce (via `a2` $\gg 5$) with a *Share 0*–only nonce expression (via `t6` $\ll 25$). Hence, the key being fixed, it leaks part of the nonce, due to the overwrite of both *shares* of the nonce.

Leak 2: Instruction at PC `0x17d8` (Index 2683). We attribute the observed effect to the destination register of the previous PC, `0x17d4` (index 2682) where register `s11` is updated. Immediately before, at `0x17c4`, `s11` held `t6` $\ll$ `25`; at `0x17d4` is is overwritten with `t1` $\ll$ `25`. Expanding the register contents:

PC 0x17c4: $[((N1^0 \oplus N3^0) \cdot \tilde{R}OL5(N0^0)) \oplus ((N1^0 \oplus N3^0) \cdot N1^0) \ll 25]$

PC 0x17d4: $[((N1^1 \oplus N3^1) \cdot \tilde{R}OL5(N0^1)) \oplus ((N1^1 \oplus N3^1) \cdot N1^1) \ll 25]$.

This transition overwrites in `s11` the *Share 0*-derived of nonce with that of a *Share 1*-derived value. Because both assignments shift left by 25, only bits 25..31 of `s11` can toggle; equivalently, the HD depends solely on the 7 LSBs of

[8] Tables 7 and 8 in Appendix D shows detailed register contents.

Table 5: TVLA and `ISALeak` evaluation results for Masked Ascon (1st Round)

PicoRV32 (over 6500 samples)

		`ISALeak`	
		Leak	No Leak
TVLA	Leak	62	296
	No Leak	35	6107

Ibex (over 800 samples)

		`ISALeak`	
		Leak	No Leak
TVLA	Leak	22	73
	No Leak	3	702

($t6 \oplus t1$). This masked Ascon implementation uses de-interleaving, hence the 7 LSBs get mapped to *nonce_word*0*_planeO* (bits 55, 57, 59, 61, 63, 1, 3).

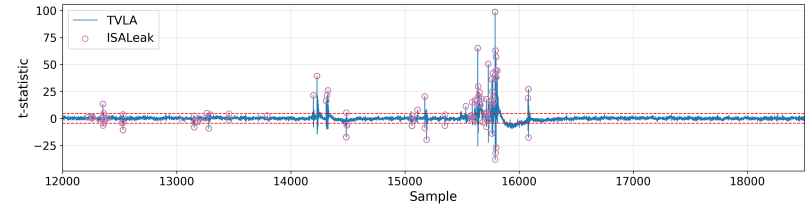
5.3 Validation

In this step, we correlate the measured power traces with the selection of *sensitive variables* and with the HW of such *sensitive variables*. Figure 8 shows annotated TVLA traces obtained from PicoRV32 and Ibex core, where the pink circles indicate samples where `ISALeak` successfully infers the root-cause of the leaks at the architectural level. Table 5 provides an overwiew of the total number of peaks detected by TVLA and the number of peaks that can be accounted for using `ISALeak`. On the PicoRV32 core, the first round of the masked Ascon implementation spans roughly 6500 samples, whereas on Ibex it spans only about 800 samples. The number of leakage samples that can be accounted for depends on the set of data interactions obtained in the initial step of `ISALeak`.

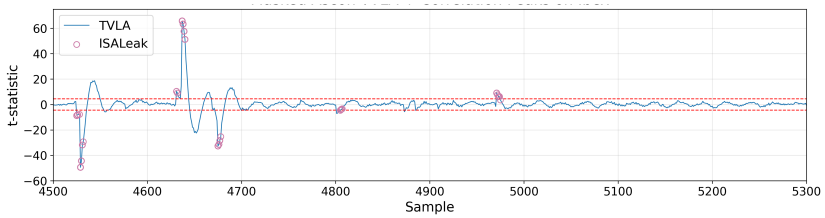
Table 5 shows that the ISALeak resolves 17% (62 out of 358) and 23% (22 out of 95) of TVLA leaks for PicoRV32 and Ibex, respectively. Furthermore, from 12 000 to 14 000 (PicoRV32) and from 4800 to 5000 (Ibex) several sample points depicts leakage as per ISALeak caused due to unintended interactions between masked shares of *sensitive variables*, which TVLA fails to capture.

6 Discussion

Given the emphasis on microarchitectural leakage, does architecture-level analysis still matter? Although many side-channel leaks manifest at the microarchitectural level, their root causes often lie at the ISA level, making architecture-level analysis necessary. Compiler decisions, such as instruction scheduling and register allocation, directly shape the signals that propagate through hardware, meaning ISA-level behavior can introduce side-channel leaks that amplify downstream microarchitectural leakage. Consistent with previous work [22], which traces many microarchitectural leaks back to causes at the ISA-level, our results from `ISALeak` show that a substantial fraction of observed leaks, 45% (PicoRV32) and 22% (Ibex), are caused by unwanted register-level interactions. Even for masked Ascon, `ISALeak` detects 17%(PicoRV32)–23%(Ibex) of leaks and reveals unintended interactions between masked shares.



(a) PicoRV32 Core



(b) Ibex Core

Fig. 8: Masked Ascon (1st Round): TVLA and Correlation (GCC v10.2, `-Os`)

Why is correlation an appropriate detection metric? The use of correlation as a detection metric is motivated by its *linear invariance property*, which guarantees that linear transformations of the simulated power values do not alter the measured dependence between data and leakage. In the context of `ISALeak`, the simulated power at each instruction is modeled as the sum of Hamming-weight or Hamming-distance contributions from all active registers. Among these, only the destination register and the program counter (PC) typically change between consecutive instructions, while the remaining registers retain constant values across different inputs. These constant components merely add a fixed offset to the simulated power and therefore do not affect the correlation outcome. Consequently, the Pearson correlation remains robust to such additive or scaling effects, ensuring that the detected dependencies truly reflect data-dependent leakage rather than artifacts of the abstraction. This property makes correlation-based analysis particularly effective in identifying genuine leakage sources, even when the traces are derived from high-level architectural simulations rather than detailed physical measurements.

How effective is TVLA at detecting side-channel leakage? As a moment-based statistical test, it is known [33,34] that TVLA may suffer from false negatives and fail to capture subtle data-dependent dependencies. In contrast, correlation reveals additional leakage that TVLA overlooks. For masked AES, correlation identifies significant data dependence in 12.7% of samples on PicoRV32 and 23.4% on Ibex. For masked Ascon, it still detects measurable leakage in 0.56% (PicoRV32) and 0.42% (Ibex) of samples. The difference with TVLA effectiveness between these cases is largely explained by the number of variables considered during the correlation, which influences the sensitivity of the detection process.

How can `ISALeak` be applied to a different implementations of the same cryptographic algorithm? Applying `ISALeak` to different implementations of the same cryptographic algorithm follows the same *identify–attribute–validate* workflow, with varying degrees of adaptation required at each stage. The *identify* requires changes in byte interactions that violate the independence property mandated by the masking scheme. The *attribute* step is the most sensitive to implementation changes, as variations in the compiler version and optimization level affect register allocation and instruction scheduling, necessitating a re-execution of attribution to correctly map leakage points to architectural sources. In contrast, the *validate* step can be applied directly across different implementations, since it operates on the derived leakage hypotheses and measured traces without relying on implementation-specific instruction layouts.

What strategies effectively mitigate the identified leaks? `ISALeak` enables evaluation of full protected implementations for value and transition-based leaks. These data-dependent leaks can be mitigated using countermeasures such as introducing fresh randomness or updating the register allocation strategy to ensure the independence assumption mandated by masking. A recent work in this direction is proposed in [36], which proposes a register allocation strategy based on the CPU-independent leakage model for ARM targets. Our analysis reveals that several leaks originate during intermediate data handling, either in the preparatory steps preceding an operation or at the junction between two operations. A practical countermeasure in these cases is zeroizing registers after sensitive computations, preventing residual values from being reused. Nevertheless, this approach brings design challenges: determining when zeroization should occur, how to automate its insertion without compromising correctness.

Can the attribution step be automated? Parts of the framework, such as backtracking register contents, are already partially automated. Extending this automation to fully support systematic backtracking and root-cause attribution remains an important direction for future work.

7 Conclusion

In this work, we investigated the root causes of side-channel leakage in full cryptographic implementations targeting RISC-V architectures. We propose `ISALeak` that enables instruction-level attribution of leaks, enabling fine-grained root-cause analysis for full implementations. Our results demonstrate that functionally correct masked cryptographic code can be vulnerable to side-channels due to register reuse, instruction reordering, or sensitive data transitions unintentionally introduced by the compiler. Case studies on masked AES and Ascon across multiple GCC versions and optimization levels show that side-channel–unaware compilation introduces exploitable leakage even in protected implementations. Measurements on an ASIC RISC-V core confirm that these leaks remain observable despite microarchitectural effects and noise. Together, these findings indicate that leakage is not purely microarchitectural and that ISA-level analysis provides an effective first line of defense for early detection and mitigation.

A Assembly code of `remask` Function (Masked AES)

This section presents the assembly code snippet of `remask` function generated for different optimization levels and varying compiler version. The instructions are annotated with the operations performed per byte of the state matrix (s) and the contents of the registers. Listing 3 corresponds to `remask` function for optimization level `-O1` and GCC version 13.2, while Listing 4 depicts instructions for optimization level `-O2` for GCC version 14.2.

Listing 3 `[AES M-O1]` Remask function, GCC v13.2, extracted from Ghidra

```
1    // remask function call initialize  a1=m1,a2=m2,a3=m3,a4=
         m',a5=m',a6=m',a7=m'
2  0xaf4     mv       t1, a0
3  0xaf8     addi     a0, a0, 0x10
4  0xafc     xor      a1, a1, a5      // a1 contains  m1 ⊕ m'
5  0xb00     xor      a2, a2, a6      // a2 contains  m2 ⊕ m'
6  0xb04     xor      a3, a3, a7      // a3 contains  m3 ⊕ m'
7  0xb08     lbu      a5, 0x0(sp)
8  0xb0c     xor      a4, a4, a5      // a4 contains  m4 ⊕ m'
9     // LAB_00000b10
10 0xb10     lbu      t3, 0x0(t1)     // load  s[0] ⊕ m'
11 0xb14     xor      t3, t3, a1      // t3 contains  s[0] ⊕ m1
12 0xb18     sb       t3, 0x0(t1)     // store s[0] ⊕ m1 as s[0]
13 0xb1c     lbu      t3, 0x1(t1)     // load  s[1] ⊕ m'
14 0xb20     xor      t3, t3, a2      // t3 contains s[1] ⊕ m2
15 0xb24     sb       t3, 0x1(t1)     // store s[1] ⊕ m2 as s[1]
16 0xb28     lbu      t3, 0x2(t1)     // load  s[2] ⊕ m'
17 0xb2c     xor      t3, t3, a3      // t3 contains s[2] ⊕ m3
18 0xb30     sb       t3, 0x2(t1)     // store s[2] ⊕ m3 as s[2]
19 0xb34     lbu      t3, 0x3(t1)     // load  s[3] ⊕ m'
20 0xb38     xor      t3, t3, a4      // t3 contains s[3] ⊕ m4
21 0xb3c     sb       t3, 0x3(t1)     // store s[3] ⊕ m4 as s[3]
22 0xb40     addi     t1, t1, 0x4     // calc index of next col
23 0xb44     bne      t1, a0, LAB_00000b10  // jump to start
24 0xb48     ret
```

B Correlation Analysis of Masked AES using Simulated Traces

We provide an exhaustive list of byte interactions that result in statistically significant correlation values for masked AES in Table 6.

Listing 4 `[AES M-O2]` Remask function, GCC v14.2, extracted from Ghidra

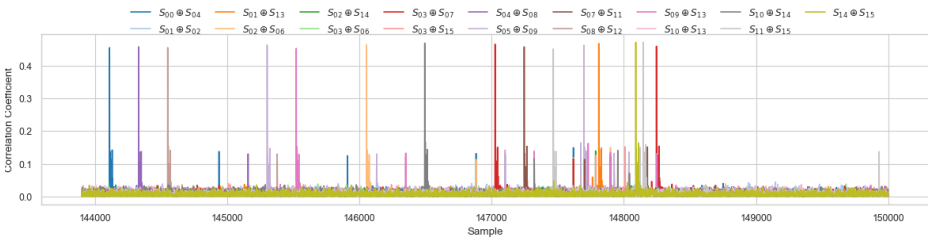
```
1   LAB_000011b8
2  0x11b8    lbu     a0, 0x0(a5)     // load  s[0] ⊕ m'
3  0x11bc    lbu     a2, 0x1(a5)     // load  s[1] ⊕ m'
4  0x11c0    lbu     a3, 0x2(a5)     // load  s[2] ⊕ m'
5  0x11c4    lbu     a4, 0x3(a5)     // load  s[3] ⊕ m'
6  0x11c8    xor     a0, t6, a0    // t6 has m0 ⊕ m',a0 has
       s[0] ⊕ m1
7  0x11cc    xor     a2, t0, a2    // t0 has m1 ⊕ m',a2 has
       s[1] ⊕ m2
8  0x11d0    xor     a3, t2, a3    // t2 has m2 ⊕ m',a3 has
       s[2] ⊕ m3
9  0x11d4    xor     a4, a1, a4    // a1 has m4 ⊕ m',a4 has
       s[3] ⊕ m4
10 0x11d8    sb      a0, 0x0(a5)   // store s[0] ⊕ m1 as s[0]
11 0x11dc    sb      a2, 0x1(a5)   // store s[1] ⊕ m2 as s[1]
12 0x11e0    sb      a3, 0x2(a5)   // store s[2] ⊕ m3 as s[2]
13 0x11e4    sb      a4, 0x3(a5)   // store s[3] ⊕ m4 as s[3]
14 0x11e8    addi    a5, a5, 0x4   // cal index of next col
15 0x11ec    bne     a5, t4, LAB_000011b8 // jump to start
16 0x11f0    mv      a5, s0
```

C Experimental Evaluation Results for Masked AES on measures Traces

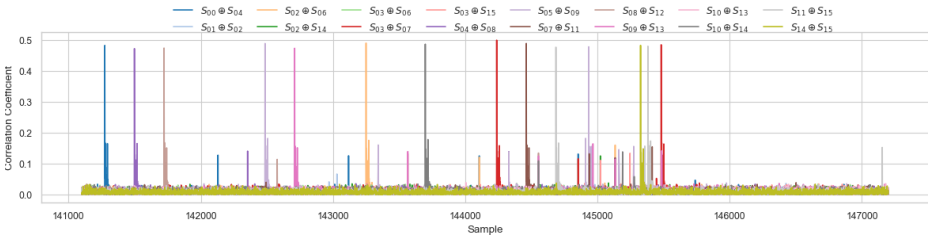
The experiments were performed on two RISC-V cores: PicoRV32 and Ibex. We present the correlation results and annotated TVLA results for the remaining optimization levels and compiler versions in this section.

Table 6: Byte Interactions in masked AES resulting in correlation peaks with 5 000 traces

Opt. Level	GCC Ver.	ShiftRows	Remask	MixColumns
-O0	12.2 13.2 14.2	$s_5 \oplus s_9$, $s_9 \oplus s_{13}$, $s_{13} \oplus s_1$,$s_1 \oplus s_{10}$, $s_{10} \oplus s_2$, $s_2 \oplus s_{14}$,$s_{14} \oplus s_6$, $s_6 \oplus s_{15}$, $s_{15} \oplus s_{11}$, $s_{11} \oplus s_7$, $s_7 \oplus s_3$	–	–
-O1	12.2 13.2 14.2	$s_5 \oplus s_9$, $s_9 \oplus s_{13}$, $s_{13} \oplus s_{10}$, $s_{10} \oplus s_{14}$, $s_{14} \oplus s_{15}$, $s_{15} \oplus s_{11}$, $s_{11} \oplus s_7$, $s_1 \oplus s_2$, $s_2 \oplus s_6$, $s_6 \oplus s_3$	–	$s_0 \oplus s_4$, $s_4 \oplus s_8$, $s_8 \oplus s_{12}$
-O2	12.2	$s_{15} \oplus s_{11}$	$s_6 \oplus s_0$, $s_{15} \oplus s_5$, $s_{11} \oplus s_{10}$, $s_7 \oplus s_{15}$	$s_{12} \oplus s_0$, $s_1 \oplus s_5$, $s_{15} \oplus s_3$, $s_3 \oplus s_7$, $s_7 \oplus s_{11}$
	13.2	$s_{15} \oplus s_{11}$	$s_7 \oplus s_{15}$, $s_6 \oplus s_0$ $s_{15} \oplus s_5$, $s_{11} \oplus s_{10}$	$s_0 \oplus s_4$, $s_4 \oplus s_8$, $s_8 \oplus s_{12}$
	14.2	$s_{15} \oplus s_7$	$s_{14} \oplus s_0$, $s_3 \oplus s_5$, $s_7 \oplus s_{10}$, $s_{11} \oplus s_{15}$	–
-O3	12.2	$s_0 \oplus s_8$, $s_5 \oplus s_9$, $s_2 \oplus s_{14}$, $s_8 \oplus s_{12}$, $s_2 \oplus s_6$		
	13.2	$s_8 \oplus s_{12}$, $s_2 \oplus s_{10}$, $s_1 \oplus s_9$		
	14.2	$s_2 \oplus s_{1012}$, $s_1 \oplus s_3$		
-Os	12.2 13.2	$s_5 \oplus s_9$, $s_9 \oplus s_{13}$, $s_{13} \oplus s_{10}$, $s_{10} \oplus s_{14}$, $s_{14} \oplus s_{15}$, $s_{15} \oplus s_{11}$, $s_{11} \oplus s_7$, $s_1 \oplus s_2$, $s_2 \oplus s_6$, $s_6 \oplus s_3$	–	$s_0 \oplus s_4$, $s_4 \oplus s_8$, $s_8 \oplus s_{12}$, $s_1 \oplus s_5$, $s_5 \oplus s_9$, $s_9 \oplus s_{13}$, $s_2 \oplus s_6$, $s_6 \oplus s_{10}$, $s_{10} \oplus s_{14}$, $s_3 \oplus s_7$, $s_7 \oplus s_{11}$, $s_{11} \oplus s_{15}$
	14.2	$s_5 \oplus s_9$, $s_9 \oplus s_{13}$, $s_{13} \oplus s_{10}$, $s_{10} \oplus s_{14}$, $s_{14} \oplus s_{15}$, $s_{15} \oplus s_{11}$, $s_{11} \oplus s_7$, $s_1 \oplus s_2$, $s_2 \oplus s_6$, $s_6 \oplus s_3$	–	$s_0 \oplus s_4$, $s_4 \oplus s_8$, $s_8 \oplus s_{12}$, $s_2 \oplus s_6$, $s_6 \oplus s_{10}$, $s_{10} \oplus s_{14}$ $s_3 \oplus s_7$, $s_7 \oplus s_{11}$, $s_{11} \oplus s_{15}$

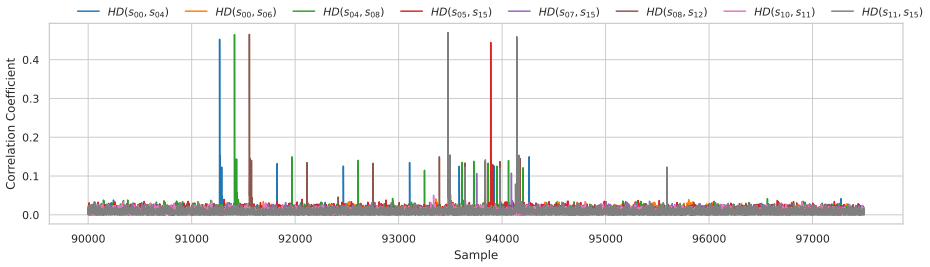


(a) v13.2

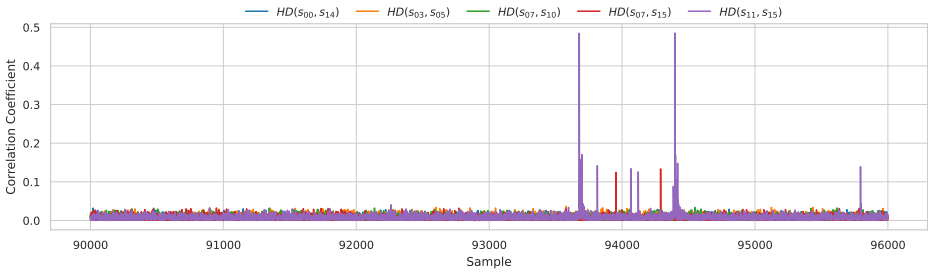


(b) v14.2

Fig. 9: Correlation results for masked AES (-Os) over 10 000 power traces.

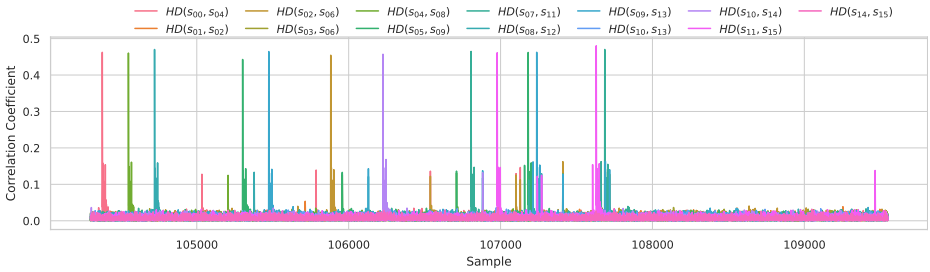


(a) v13.2

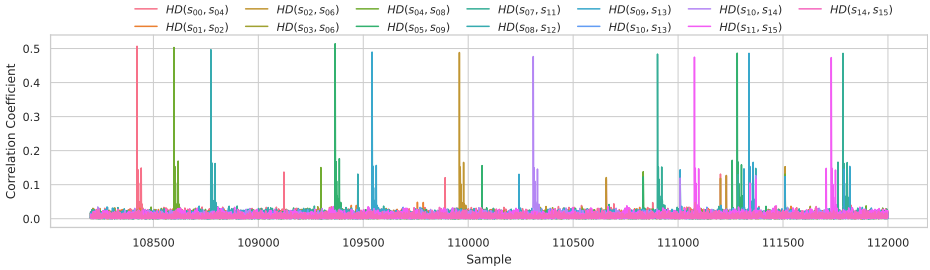


(b) v14.2

Fig. 10: Correlation results for masked AES (`-O2`) over 10 000 power traces.



(a) v13.2



(b) v14.2

Fig. 11: Correlation results for masked AES (`-O1`) over 10 000 power traces.

D Attribution of Leaks in `LINEAR` operation of `P2` function in Masked Ascon

Table 7: Register state during the execution of `LINEAR` at `−O1` (GCC v10.2). The rows correspond to the registers used in `LINEAR`. Each column corresponds to one instruction specified by the program counter values. The expression in each cell indicates the first time the value is loaded into the register; $ROL5(x) = (x \ll 5)|(x \gg 27)$; $ROR5(x) = (x \gg 5)|(x \ll 27)$; $\ll$ & $\gg$ denote logical left-shift and logical right-shift, respectively. Let $A = K0^1 \oplus K2^1$, $B = K1^1 \oplus K3^1$, $C = N1^0 \oplus N3^0$

	Pre-loaded	0x1498	0x14a4	0x16e8	0x16f4	0x1788	0x17c4
a2	$K1^1$			$K1^1 \oplus (N1^1 \cdot \tilde{ROL5}(A))$	$K1^1 \oplus (N1^1 \cdot \tilde{ROL5}(A)) \oplus (N1^1 \cdot B)$		
t6	$N0^0 \ll 5$	$C \cdot \tilde{ROL5}(N0^0)$	$(C \cdot \tilde{ROL5}(N0^0)) \oplus (C \cdot N1^0)$				
s11	$K0^1 \oplus (N0^1 \cdot \tilde{A}) \oplus (N0^1 \cdot ROR5(B)) \ll 5$					$K1^1 \oplus (N1^1 \cdot \tilde{ROL5}(A)) \oplus (N1^1 \cdot B) \gg 5$	$(C \cdot \tilde{ROL5}(N0^0)) \oplus (C \cdot N1^0) \ll 25$

Table 8: Register state during the execution of `LINEAR` at `−O1` (GCC v10.2); $ROL5(x) = (x \ll 5)|(x \gg 27)$; $\ll$ & $\gg$ denote logical left-shift and logical right-shift, respectively. Let $A = K0^1 \oplus K2^1$, $B = K1^1 \oplus K3^1$, $C = N1^0 \oplus N3^0$, $D = N1^1 \oplus N3^1$

	Pre-loaded	0x1498	0x14a4	0x1658	0x1664	0x17c4	0x17d4
t6	$N0^0 \ll 5$	$C \cdot \tilde{ROL5}(N0^0)$	$(C \cdot \tilde{ROL5}(N0^0)) \oplus (C \cdot N1^0)$				
t1	$\neg N0' \ll 5$			$D \cdot \tilde{ROL5}(N0^1)$	$(D \cdot \tilde{ROL5}(N0^1)) \oplus (D \cdot N1^1)$		
s11	$K1^1 \oplus (N1^1 \cdot \tilde{ROL5}(A)) \oplus (N1^1 \cdot B) \gg 5$					$(C \cdot \tilde{ROL5}(N0^0)) \oplus (C \cdot N1^0) \ll 25$	$(D \cdot \tilde{ROL5}(N0^1)) \oplus (D \cdot N1^1) \ll 25$

References

1. Adhikary, A., Basurto-Becerra, A., Batina, L., Buhan, I., Chatterjee, D., van Hoek, S., Gonzalez, E.S.: ARCHER: architecture-level simulator for side-channel analysis in RISC-V processors. IACR Cryptol. ePrint Arch. p. 1866 (2024), https://eprint.iacr.org/2024/1866
2. Amiot, N., Meunier, Q.L., Heydemann, K., Encrenaz, E.: aleakator: HDL mixed-domain simulation for masked hardware & software formal verification. IACR Cryptol. ePrint Arch. p. 2193 (2025), https://eprint.iacr.org/2025/2193
3. Arora, V., Buhan, I., Perin, G., Picek, S.: A tale of two boards: On the influence of microarchitecture on side-channel leakage. In: Grosso, V., Pöppelmann, T. (eds.) Smart Card Research and Advanced Applications - 20th International Conference, CARDIS 2021, Lübeck, Germany, November 11-12, 2021, Revised Selected Papers. Lecture Notes in Computer Science, vol. 13173, pp. 80–96. Springer (2021). https://doi.org/10.1007/978-3-030-97348-3_5
4. Balasch, J., Gierlichs, B., Grosso, V., Reparaz, O., Standaert, F.: On the cost of lazy engineering for masked software implementations. In: Joye, M., Moradi, A. (eds.) Smart Card Research and Advanced Applications - 13th International Conference, CARDIS 2014, Paris, France, November 5-7, 2014. Revised Selected Papers. Lecture Notes in Computer Science, vol. 8968, pp. 64–81. Springer (2014). https://doi.org/10.1007/978-3-319-16763-3_5
5. Bazangani, O., Iooss, A., Buhan, I., Batina, L.: ABBY: automating leakage modelling for side-channel analysis. In: Zhou, J., Quek, T.Q.S., Gao, D., Cárdenas, A.A. (eds.) Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2024, Singapore, July 1-5, 2024. ACM (2024). https://doi.org/10.1145/3634737.3637665
6. Bertoni, G., Daemen, J., Peeters, M., Assche, G.V.: Sponge functions. In: ECRYPT Hash Workshop 2007 (2007), https://keccak.team/files/SpongeFunctions.pdf
7. Buhan, I., Batina, L., Yarom, Y., Schaumont, P.: Sok: Design tools for side-channel-aware implementations. In: Suga, Y., Sakurai, K., Ding, X., Sako, K. (eds.) ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June 2022. pp. 756–770. ACM (2022). https://doi.org/10.1145/3488932.3517415
8. Chari, S., Jutla, C.S., Rao, J.R., Rohatgi, P.: Towards sound approaches to counteract power-analysis attacks. In: Wiener, M.J. (ed.) Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings. Lecture Notes in Computer Science, vol. 1666, pp. 398–412. Springer (1999). https://doi.org/10.1007/3-540-48405-1_26
9. Chowdhury, S., Mishra, N., Bhattacharya, S., Mukhopadhyay, D.: Architectural leakage analysis of masked cryptographic software on RISC-V cores. IACR Cryptol. ePrint Arch. p. 1944 (2025), https://eprint.iacr.org/2025/1944
10. Daemen, J., Rijmen, V.: The Design of Rijndael: AES - The Advanced Encryption Standard. Information Security and Cryptography, Springer (2002). https://doi.org/10.1007/978-3-662-04722-4
11. Dobraunig, C., Eichlseder, M., Mendel, F., Schläffer, M.: Ascon v1.2: Lightweight authenticated encryption and hashing. J. Cryptol. **34**(3), 33 (2021). https://doi.org/10.1007/S00145-021-09398-9
12. Faust, S., Grosso, V., Pozo, S.M.D., Paglialonga, C., Standaert, F.: Composable masking schemes in the presence of physical defaults & the robust probing model.

IACR Trans. Cryptogr. Hardw. Embed. Syst. **2018**(3), 89–120 (2018). https://doi.org/10.13154/TCHES.V2018.I3.89-120
13. Gandolfi, K., Mourtel, C., Olivier, F.: Electromagnetic analysis: Concrete results. In: Koç, Ç.K., Naccache, D., Paar, C. (eds.) Cryptographic Hardware and Embedded Systems - CHES 2001, Third International Workshop, Paris, France, May 14-16, 2001, Proceedings. Lecture Notes in Computer Science, vol. 2162, pp. 251–261. Springer (2001). https://doi.org/10.1007/3-540-44709-1_21
14. Gaspoz, J., Dhooghe, S.: Threshold implementations in software: Micro-architectural leakages in algorithms. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2023**(2), 155–179 (2023). https://doi.org/10.46586/TCHES.V2023.I2.155-179
15. Gigerl, B., Hadzic, V., Primas, R., Mangard, S., Bloem, R.: Coco: Co-design and co-verification of masked software implementations on cpus. In: Bailey, M.D., Greenstadt, R. (eds.) 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021. pp. 1469–1468. USENIX Association (2021), https://www.usenix.org/conference/usenixsecurity21/presentation/gigerl
16. Gigerl, B., Mendel, F., Schläffer, M., Primas, R.: Efficient second-order masked software implementations of ascon in theory and practice. IACR Cryptol. ePrint Arch. p. 755 (2024), https://eprint.iacr.org/2024/755
17. Goodwill, G., Jun, J., P.Rohatgi: A testing methodology for side channel resistance validation. NIST non-invasive attack testing workshop (2018)
18. Ishai, Y., Sahai, A., Wagner, D.A.: Private circuits: Securing hardware against probing attacks. In: Boneh, D. (ed.) Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2729, pp. 463–481. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_27
19. Kiaei, P., Liu, Z., Eren, R.K., Yao, Y., Schaumont, P.: Saidoyoki: Evaluating side-channel leakage in pre- and post-silicon setting. IACR Cryptol. ePrint Arch. p. 1235 (2021), https://eprint.iacr.org/2021/1235
20. Kocher, P.C.: Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In: Koblitz, N. (ed.) Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings. Lecture Notes in Computer Science, vol. 1109, pp. 104–113. Springer (1996). https://doi.org/10.1007/3-540-68697-5_9
21. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M.J. (ed.) Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings. Lecture Notes in Computer Science, vol. 1666, pp. 388–397. Springer (1999). https://doi.org/10.1007/3-540-48405-1_25
22. Liu, Z., Malnicof, A., Roy, A., Schaumont, P.: Telescope: Top-down hierarchical pre-silicon side-channel leakage assessment in system-on-chip design. In: Proceedings of the 20th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2025, Hanoi, Vietnam, August 25-29, 2025. pp. 1280–1293. ACM (2025). https://doi.org/10.1145/3708821.3736216
23. lowRISC: Lowrisc/ibex-demo-system: A demo system for ibex including debug support and some peripherals, https://github.com/lowRISC/ibex-demo-system, accessed 15-10-2023
24. Mahdion, N., Oswald, E.: Efficiently detecting masking flaws in software implementations. IACR Commun. Cryptol. **1**(3), 35 (2024). https://doi.org/10.62056/AB89KSDJA
25. Mangard, S., Oswald, E., Popp, T.: Power analysis attacks - revealing the secrets of smart cards. Springer (2007)

26. Marshall, B., Page, D., Webb, J.: MIRACLE: micro-architectural leakage evaluation A study of micro-architectural power leakage across many devices. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2022**(1), 175–220 (2022). https://doi.org/10.46586/TCHES.V2022.I1.175-220
27. McCann, D., Oswald, E., Whitnall, C.: Towards practical tools for side channel aware software engineering: 'grey box' modelling for instruction leakages. In: Kirda, E., Ristenpart, T. (eds.) 26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017. pp. 199–216. USENIX Association (2017), https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/mccann
28. Meyer, L.D., Mulder, E.D., Tunstall, M.: On the effect of the (micro)architecture on the development of side-channel resistant software. IACR Cryptol. ePrint Arch. p. 1297 (2020), https://eprint.iacr.org/2020/1297
29. Papagiannopoulos, K., Veshchikov, N.: Mind the gap: Towards secure 1st-order masking in software. In: Guilley, S. (ed.) Constructive Side-Channel Analysis and Secure Design - 8th International Workshop, COSADE 2017, Paris, France, April 13-14, 2017, Revised Selected Papers. Lecture Notes in Computer Science, vol. 10348, pp. 282–297. Springer (2017). https://doi.org/10.1007/978-3-319-64647-3_17
30. Shelton, M.A., Chmielewski, L., Samwel, N., Wagner, M., Batina, L., Yarom, Y.: Rosita++: Automatic higher-order leakage elimination from cryptographic code. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021. pp. 685–699. ACM (2021). https://doi.org/10.1145/3460120.3485380
31. Shelton, M.A., Samwel, N., Batina, L., Regazzoni, F., Wagner, M., Yarom, Y.: Rosita: Towards automatic elimination of power-analysis leakage in ciphers. In: NDSS 2021. The Internet Society (2021)
32. Shelton, M.A., Samwel, N., Batina, L., Regazzoni, F., Wagner, M., Yarom, Y.: Rosita: Towards automatic elimination of power-analysis leakage in ciphers. In: 28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021. The Internet Society (2021), https://www.ndss-symposium.org/ndss-paper/rosita-towards-automatic-elimination-of-power-analysis-leakage-in-ciphers/
33. Standaert, F.: How (not) to use welch's t-test in side-channel security evaluations. In: Bilgin, B., Fischer, J. (eds.) Smart Card Research and Advanced Applications, 17th International Conference, CARDIS 2018, Montpellier, France, November 12-14, 2018, Revised Selected Papers. Lecture Notes in Computer Science, vol. 11389, pp. 65–79. Springer (2018). https://doi.org/10.1007/978-3-030-15462-2_5, https://doi.org/10.1007/978-3-030-15462-2_5
34. Whitnall, C., Oswald, E.: A cautionary note regarding the usage of leakage detection tests in security evaluation. IACR Cryptol. ePrint Arch. p. 703 (2019), https://eprint.iacr.org/2019/703
35. YosysHQ: Yosyshq/picorv32: Picorv32 - a size-optimized risc-v cpu, https://github.com/YosysHQ/picorv32
36. Zeitschner, J., Moradi, A.: Pommes: Prevention of micro-architectural leakages in masked embedded software. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2024**(3), 342–376 (2024). https://doi.org/10.46586/TCHES.V2024.I3.342-376
37. Zeitschner, J., Müller, N., Moradi, A.: Prolead_sw probing-based software leakage detection for ARM binaries. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2023**(3), 391–421 (2023). https://doi.org/10.46586/TCHES.V2023.I3.391-421